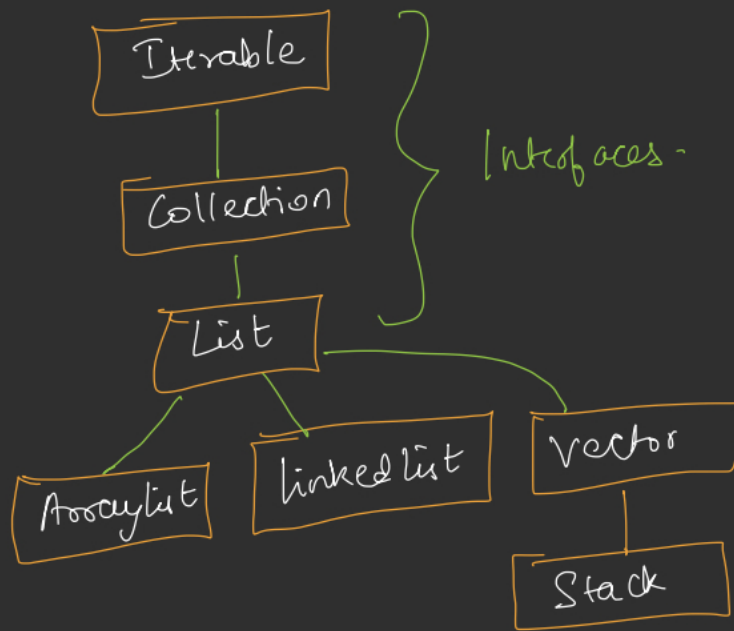
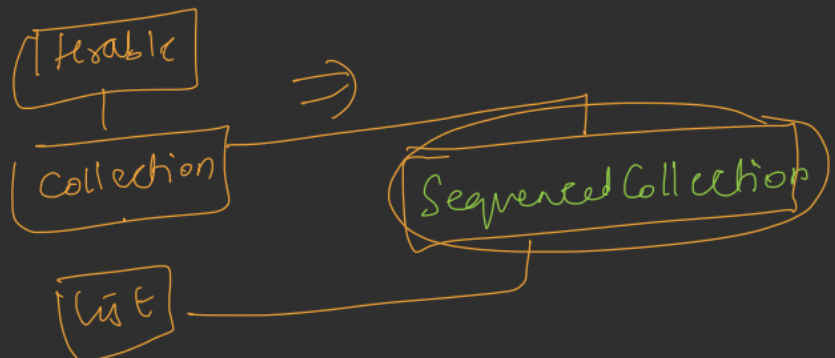




List

- Elements have a position
- Duplicates are allowed.
- Insertion Order is preserved.
- You can access element by Index.

List



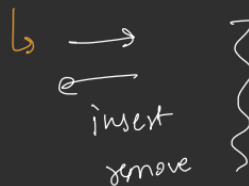
Sequenced Collection

- ↳ `getFirst()`
- ↳ `getLast()`
- ↳ `addFirst()`
- ↳ `addLast()`
- ↳ `removeFirst()`
- ↳ `removeLast()`

List

- ↳ `E get(index)`
- ↳ `set(index, E value)`
- ↳ `add(index, E value)`
- ↳ `addAll(index, Collection<? extends E>)`
- ↳ `remove(index)`

- `indexOf(Object o)`
- `lastIndexOf(Object o)`
- `listIterator()`



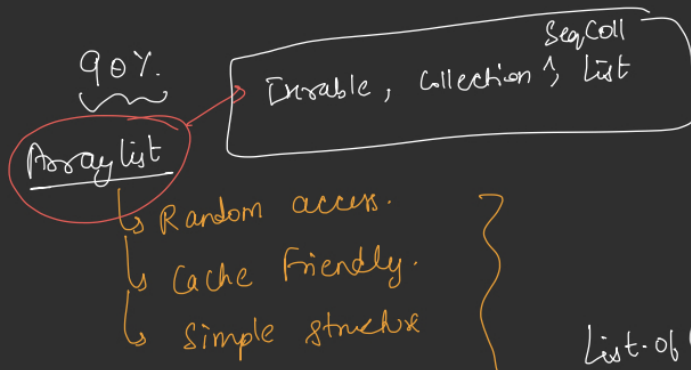
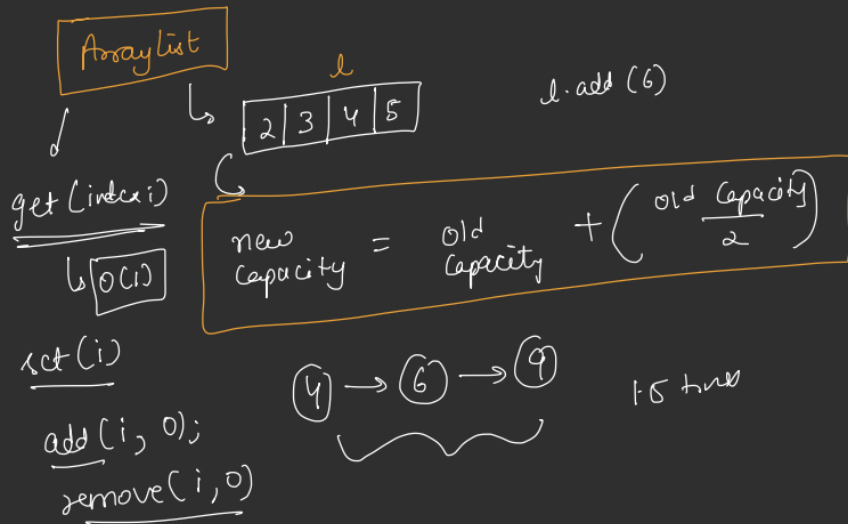
2	3	4	5
0	1	2	3

- `listIterator(int index)`

→ `of(-)`

↳ `List<Integer> l = listOf(1, 2, 3);`

→ `copyOf()`

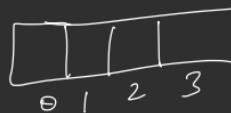


Constructors

list <Integer> l = new ArrayList <> ();
 = new ArrayList <> (10);
 = new ArrayList <> (C);

ArrayList

ensureCapacity (1);



Capacity
size

l.ensureCapacity (100);

l.trimToSize ();

Linked list (10%) ↓
→ <T>

```
class Node {  
    T data;  
    Node<T> next;  
    Node<T> prev;  
}
```

Q. add(1)
add(2)

list.get(2);

if (index < size/2)
↳ start from first

add()
remove()
add()
get()

O(N)

Queue / Stack

Vector
Stack

Legacy Classes

vector → Dynamic Array
HashMap → Hash Map
Stack → Stack Implementation using Array.

List

Queue

Set

ArrayDeque

Queue

Stack